Hyperparameter Optimization for Image Classification

A reusable methodology for picking the right network

CSCI 611 - California State University, Chico - Spring 2026

Team

Jacob - Anand - Paolo

MOTIVATION AND OVERVIEW

Context & Motivation

The goal is not food classification, it is model selection

Problem

- Many pretrained CNNs exist (ResNet, EfficientNet, MobileNet)
- Model choice is often **ad-hoc**
- Real-world constraints matter
  - Accuracy ≠ everything
  - Latency, size, and cost matter

Our Approach

- Use **food classification as a testbed**
- Compare 3 model families:
  - ResNet (performance)
  - EfficientNet (balanced)
  - MobileNet (efficiency)
- Apply **Optuna for hyperparameter optimization**
- Evaluate:
  - Accuracy
  - Model size
  - Inference speed

High-Level Framework

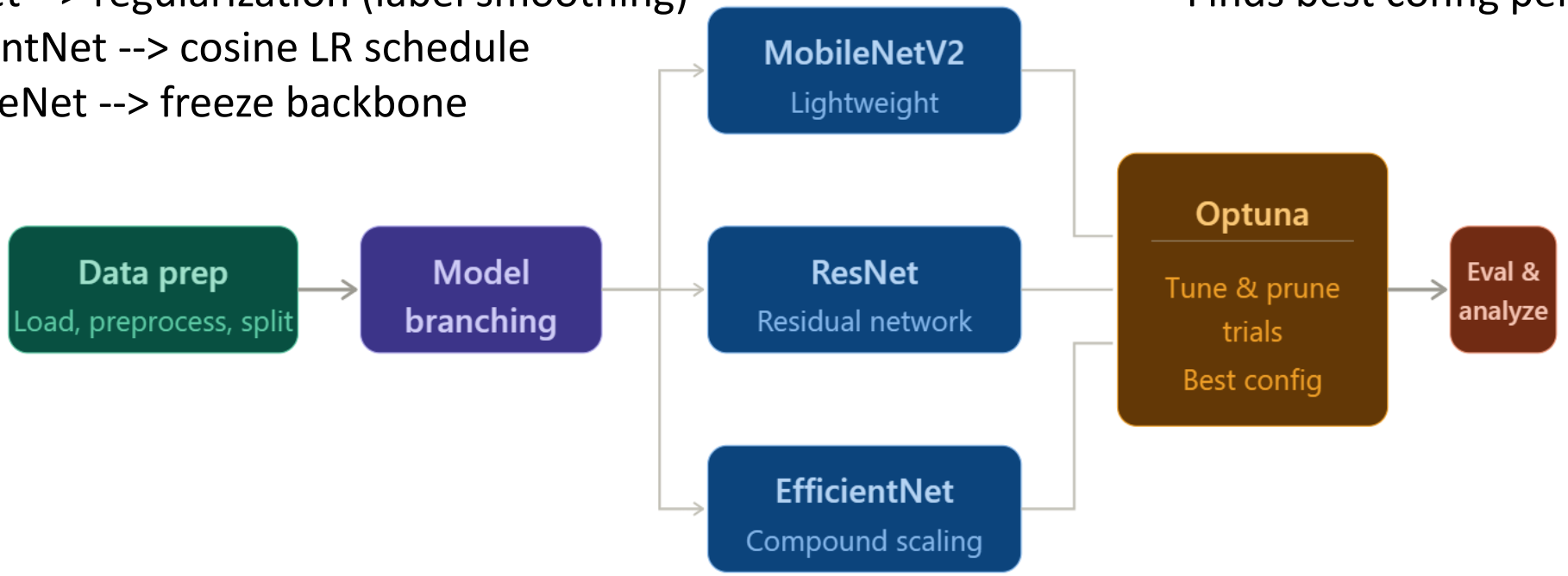
Designed for Apples-to-Apples comparison across architectures

Fair Comparison
- Fixed dataset split (YAML, seed=42)
- Same training + evaluation pipeline
- All are pretrained on ImageNet-1k

Model-Specific Tuning
- ResNet --> regularization (label smoothing)
- EfficientNet --> cosine LR schedule
- MobileNet --> freeze backbone

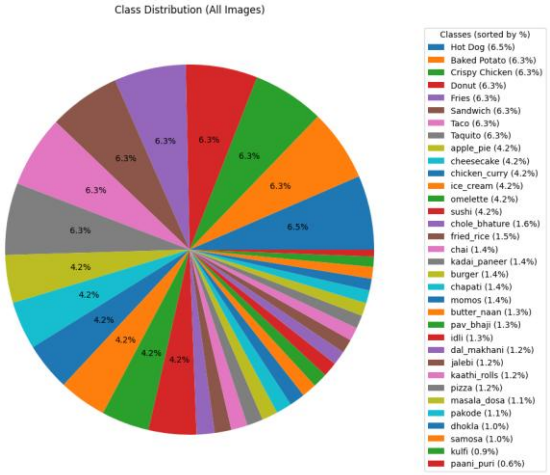
Unified Output
- One command --> report + plots
- Cross-model comparison

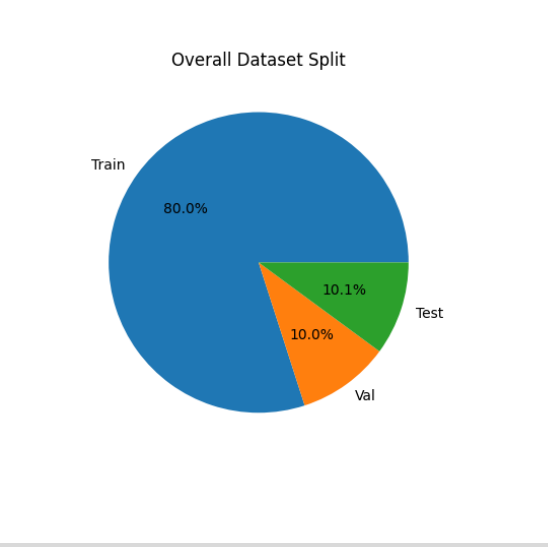
Automated Optimization
- Optuna tunes & prunes trials
- Finds best config per model



Data Split

- 23873 images of foods,
- 34 classes of food (hot dog, apple pie, donut etc.)
- 19092 of the images allocated to training
- 2377 for validation
- 2404 for testing





IMPLEMENTATION DETAILS

What is Optuna?

Overview

- Python framework which automates **Hyperparameter Optimization**
- Uses a Tree-structured Parzen Estimator (TPE)
  - Bayesian optimization algorithm which learns from past successes and failures
  - Directed approach is efficient than exhaustive grid search

Terminology

- Study: optimization based on an objective function
- Trial: a single execution of the objective function

Basic Example

```
study = optuna.create_study()  # Create a new study.
study.optimize(objective, n_trials=100)  # Invoke optimization of the objective function.
```

Building the Objective Function

Define Hyperparameter Space

```python
def objective(trial: optuna.Trial) -> float:
    # 1. Sample a hyperparameter configuration.
    common_hp = sample_common_hparams(trial, common_space)
    extra_hp = extra_space_fn(trial) if extra_space_fn is not None else {}
    hparams = {**common_hp, **extra_hp}
```

```python
class CommonSearchSpace:
    """The hyperparameters every architecture searches over.

    These mirror the standard tuning levers in image classification:
      * learning rate (log scale, by far the most important)
      * weight decay (log scale, regularization)
      * optimizer choice (SGD-momentum vs AdamW)
      * batch size (categorical, hardware-bounded)
      * dropout in the new classifier head
    """

    lr_min: float = 1e-5
    lr_max: float = 1e-2
    weight_decay_min: float = 1e-6
    weight_decay_max: float = 1e-3
    dropout_min: float = 0.0
    dropout_max: float = 0.5
    batch_sizes: tuple = (16, 32, 64)
    optimizer_choices: tuple = ("adamw", "sgd")
```

Building the Objective Function

Define Data Pipeline

```
# 2. Build the data pipeline. We rebuild loaders here (not outside
#    the objective) because batch_size is a sampled hyperparameter
#    and so it varies across trials.
manifest, train_loader, val_loader, _ = get_dataloaders(
    manifest_path=manifest_path,
    batch_size=hparams["batch_size"],
    num_workers=num_workers,
)
```

- Allows use of pytorch utilities for data segmentation and augmentation
- Batch size is a studied hyperparameter, so the objective function rebuilds the pipeline every trial

Building the Objective Function

Other Options

```
# 3a. Apply optional per-arch extras BEFORE building the optimizer
#     so frozen params are excluded from optimization.
_freeze_backbone_if_requested(built.model, hparams)

# 4. Build the optimizer using sampled lr/optimizer/weight_decay.
optimizer = build_optimizer(
    built.model,
    name=hparams["optimizer"],
    lr=hparams["lr"],
    weight_decay=hparams["weight_decay"],
)

# 4a. Loss function (honors label_smoothing if it was sampled).
criterion = _build_criterion(hparams)

# 4b. Optional LR scheduler (cosine annealing if sampled).
scheduler = _build_scheduler(optimizer, hparams, epochs=epochs_per_trial)
```

- Option to freeze CNN backbone
- Call to build pytorch optimizer object per trial
- Criterion allows for label smoothing as a parameter
- Call to build scheduler so cosine annealing lr scheduler is a sampled parameter

Building the Objective Function

Run Training

```python
# 5. Run training. If the trial gets pruned, run_training raises
#    `optuna.TrialPruned`, which Optuna catches and records.
per_trial_dir: Optional[Path] = None
if checkpoint_dir is not None:
    per_trial_dir = Path(checkpoint_dir) / f"trial_{trial.number}"

result: TrainingResult = run_training(
    model=built.model,
    train_loader=train_loader,
    val_loader=val_loader,
    epochs=epochs_per_trial,
    optimizer=optimizer,
    criterion=criterion,
    scheduler=scheduler,
    checkpoint_dir=per_trial_dir,
    checkpoint_tag=f"{arch}_trial{trial.number}",
    optuna_trial=trial,
    extra_metadata={
        "arch": arch,
        "hparams": hparams,
        "param_count": built.param_count,
    },
)

return result.best_val_acc
```

- Passes all trial hyperparameters to a function which runs pytorch training
- Logs results and returns best epoch’s validation accuracy

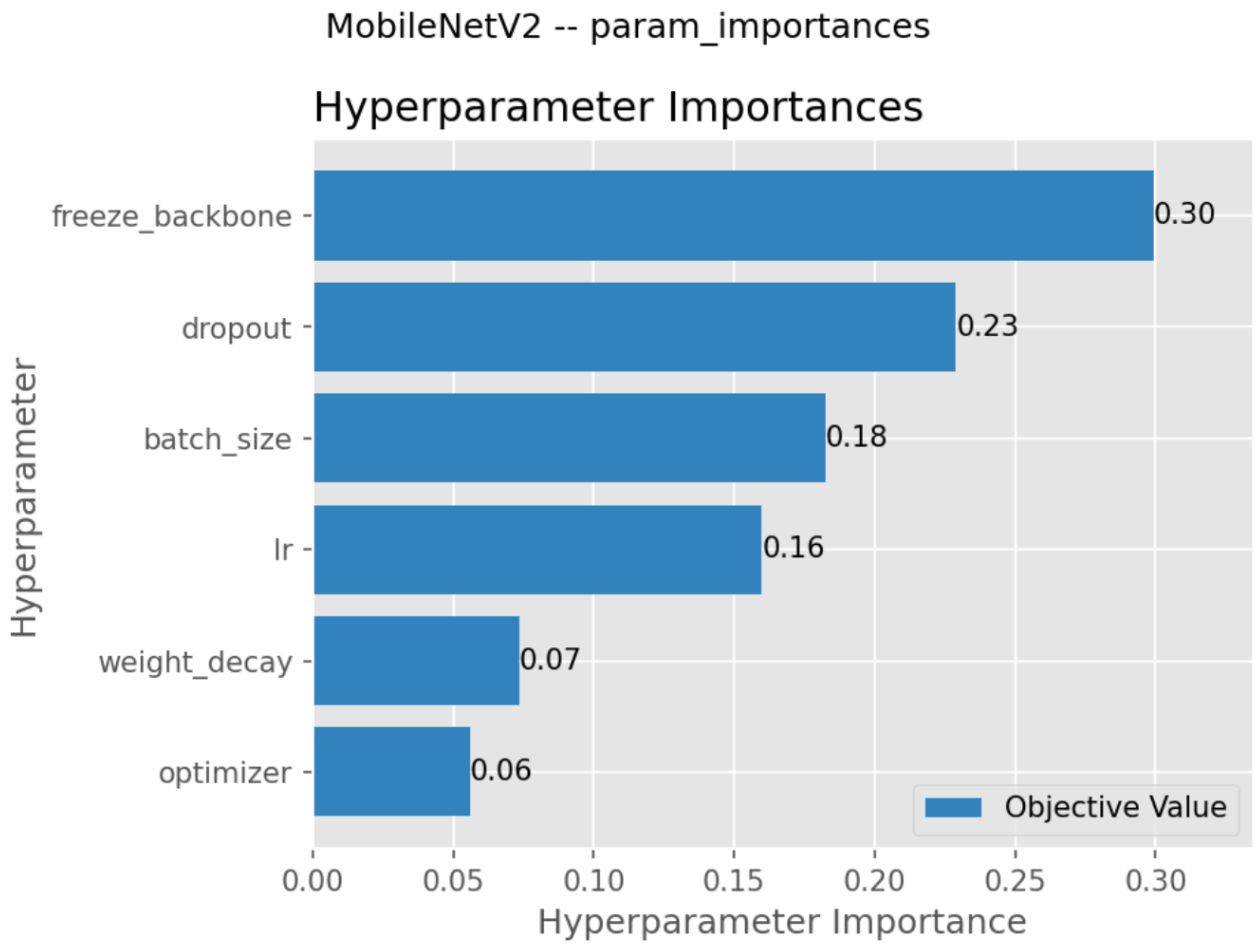
Build objective instance and run Optuna study

```
# Build the objective with ResNet-specific extra search space.
objective = make_objective(
    arch="resnet50",
    manifest_path=args.manifest,
    num_workers=args.num_workers,
    epochs_per_trial=args.epochs,
    checkpoint_dir=output_dir / "checkpoints" / "resnet50",
    common_space=CommonSearchSpace(),
    extra_space_fn=resnet_extra_space,
)

# Run the search. `n_trials` is per-call, so previous trials in the
# study DB are not rerun.
study.optimize(objective, n_trials=args.trials, gc_after_trial=True)
```

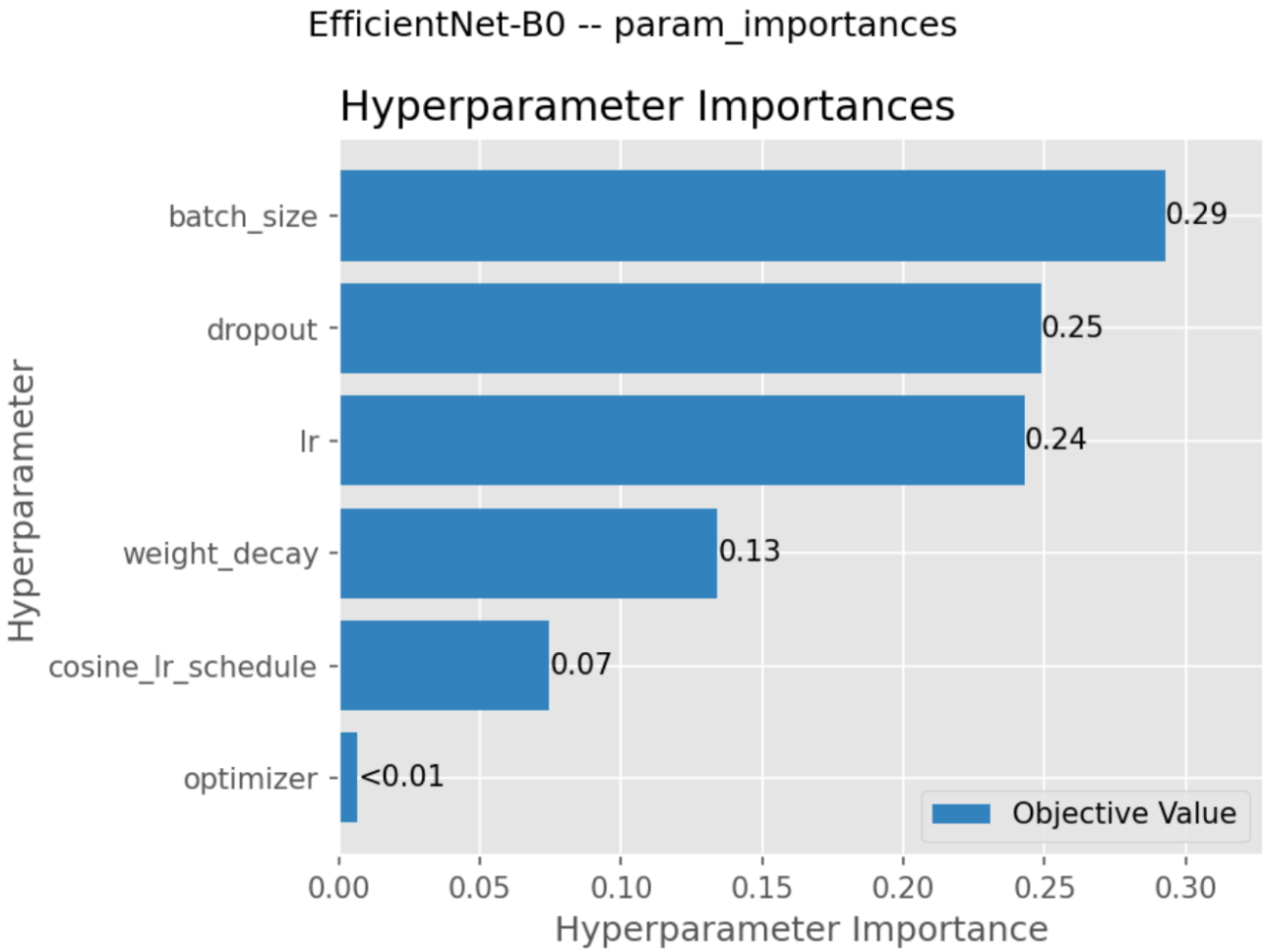
- Call factory function to build architecture-specific objective function
- Run study.optimize which is the core optuna method

Optuna Study Results: Hyperparameter Significance



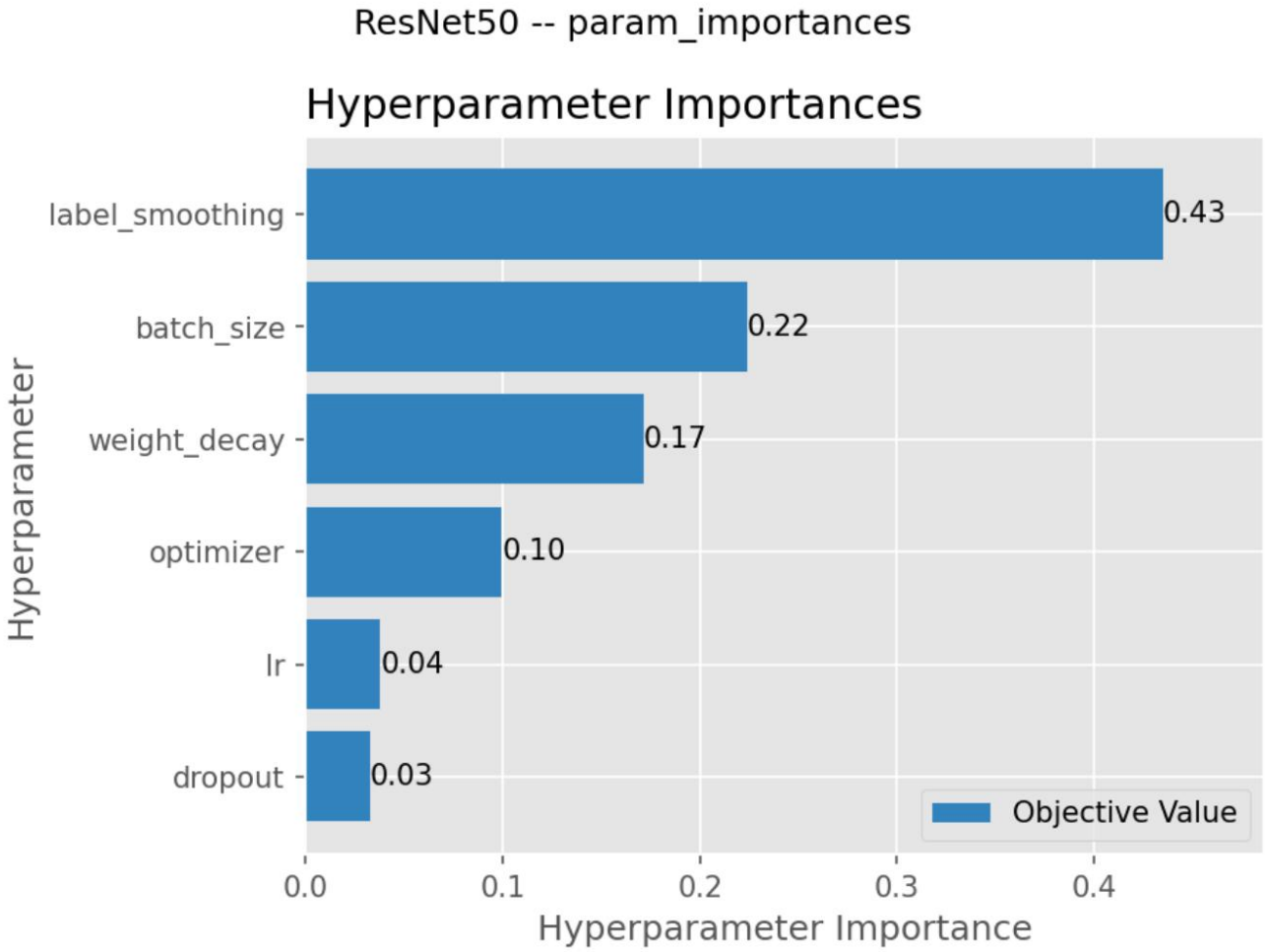

```
{
  "arch": "mobilenet_v2",
  "best_trial_number": 29,
  "best_val_acc": 0.9040807740849811,
  "best_hparams": {
    "lr": 0.002172804390993739,
    "weight_decay": 0.0006411766656125512,
    "dropout": 0.1967278050286843,
    "batch_size": 32,
    "optimizer": "sgd",
    "freeze_backbone": false
  },
  "n_trials": 31
}
```

Optuna Study Results: Hyperparameter Significance




```
{
  "arch": "efficientnet_b0",
  "best_trial_number": 5,
  "best_val_acc": 0.9221708035338663,
  "best_hparams": {
    "lr": 0.008105016126411584,
    "weight_decay": 0.00021154290797261214,
    "dropout": 0.4697494707820945 6,
    "batch_size": 64,
    "optimizer": "sgd",
    "cosine_lr_schedule": false
  },
  "n_trials": 25
}
```

Optuna Study Results: Hyperparameter Significance




```
{
  "arch": "resnet50",
  "best_trial_number": 18,
  "best_val_acc": 0.9225915018931427,
  "best_hparams": {
    "lr": 4.255433424788548e-05,
    "weight_decay": 1.9595712578988492e-05,
    "dropout": 0.1334518760377633,
    "batch_size": 16,
    "optimizer": "adamw",
    "label_smoothing": 0.09772664655950 78
  },
  "n_trials": 20
}
```

PRIMARY RESULTS

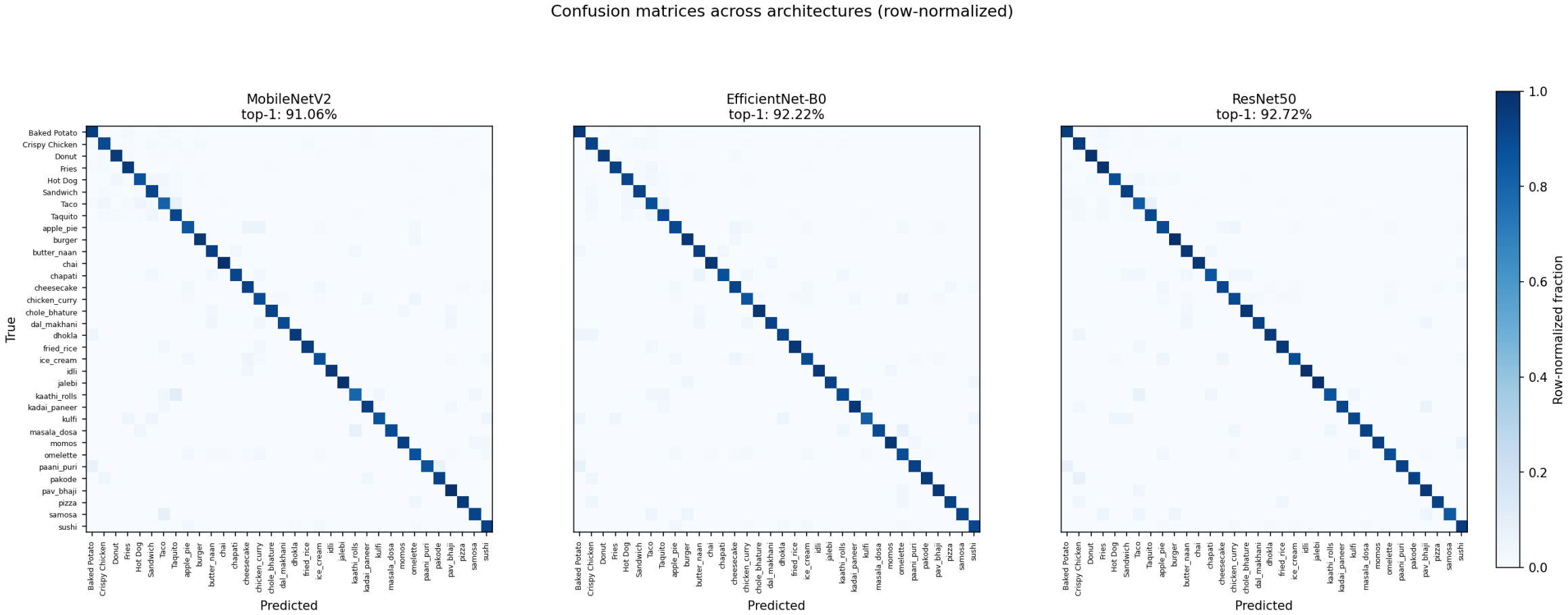
Per-class & Confusion Analysis

All models show similar confusion patterns between visually similar classes, indicating that the remaining errors are due to dataset difficulty rather than model limitations.

Taco -> Taquito (top confusion across the board)

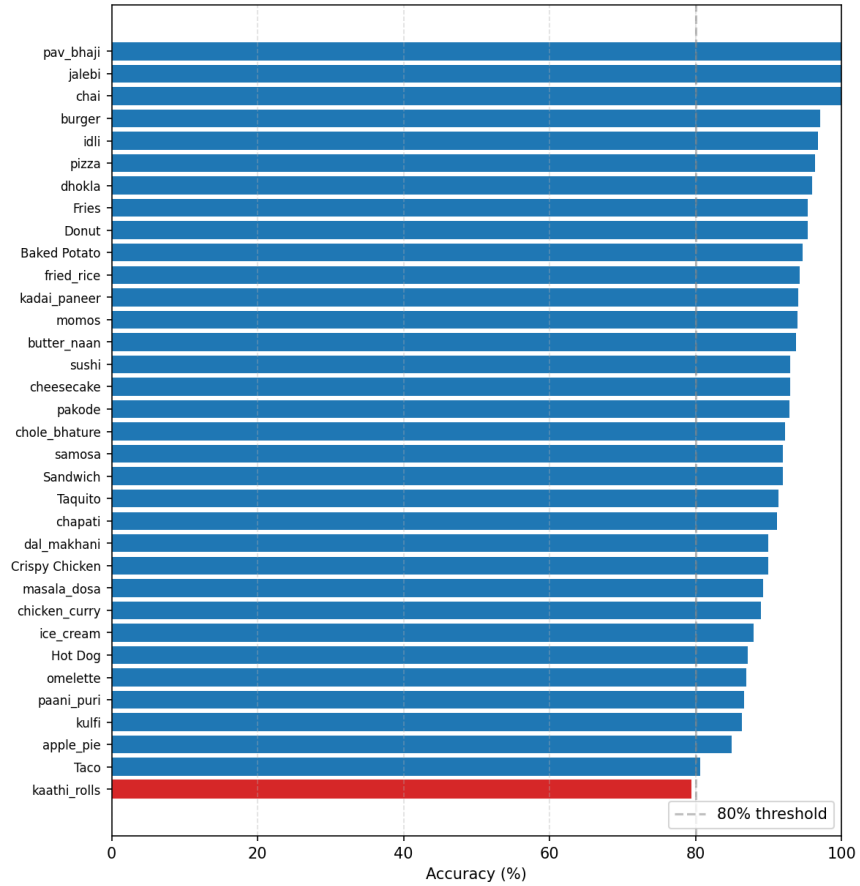
apple_pie <-> cheesecake

Top-5 saturated (>99%) -- residual error is fine-grained class confusion.

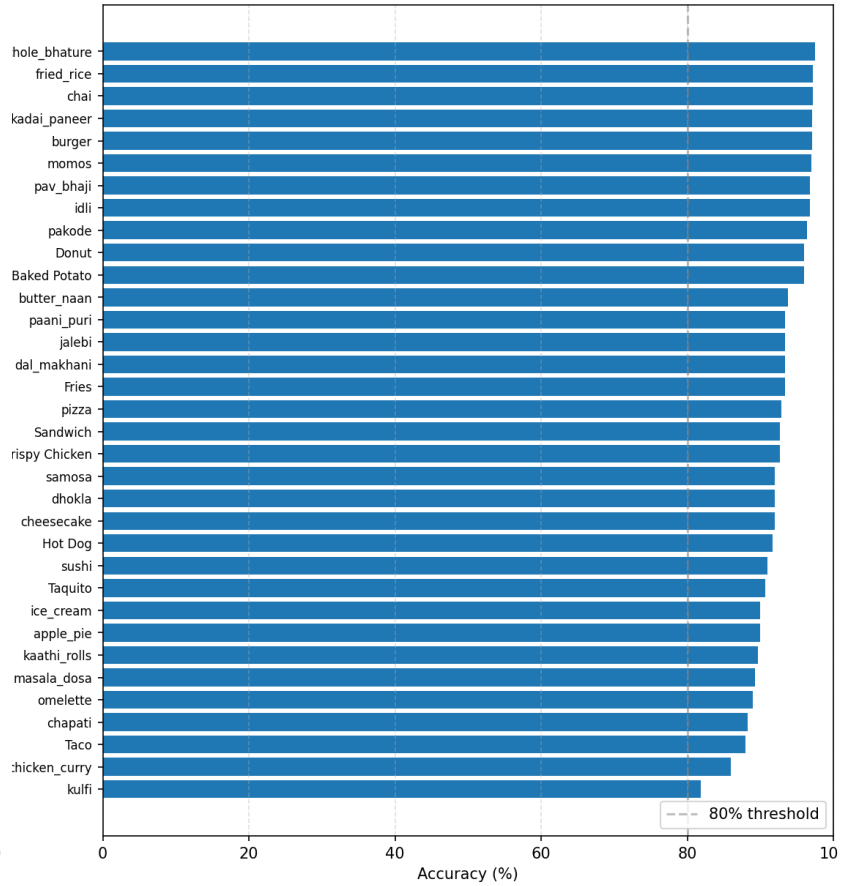


Accuracy per class

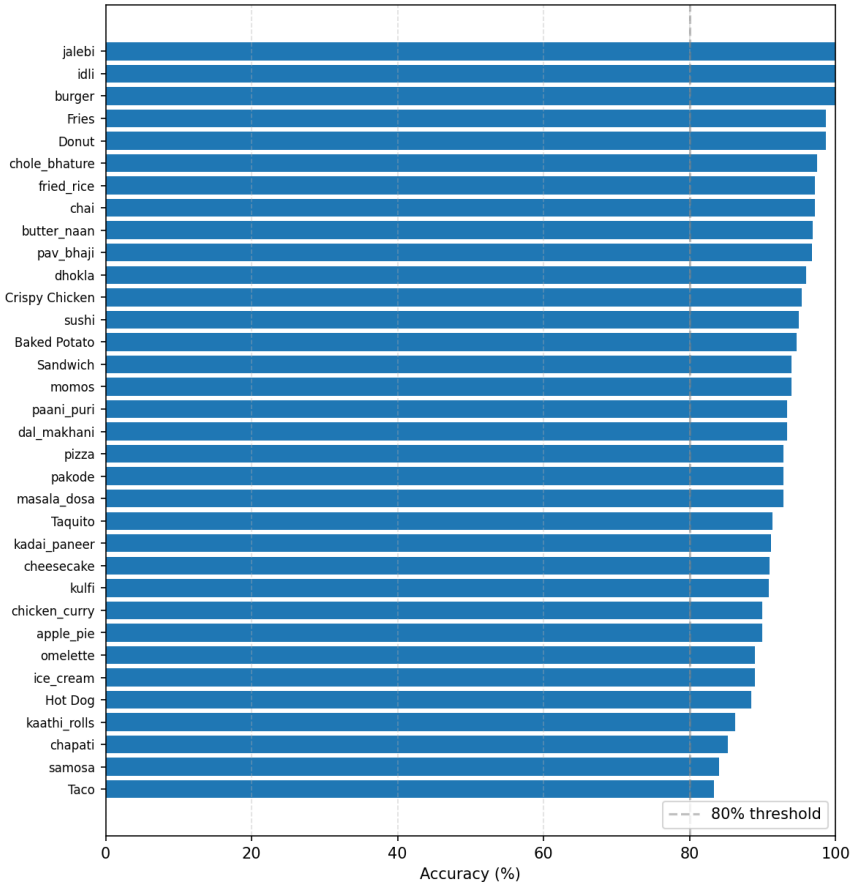
MobileNet



EfficientNet

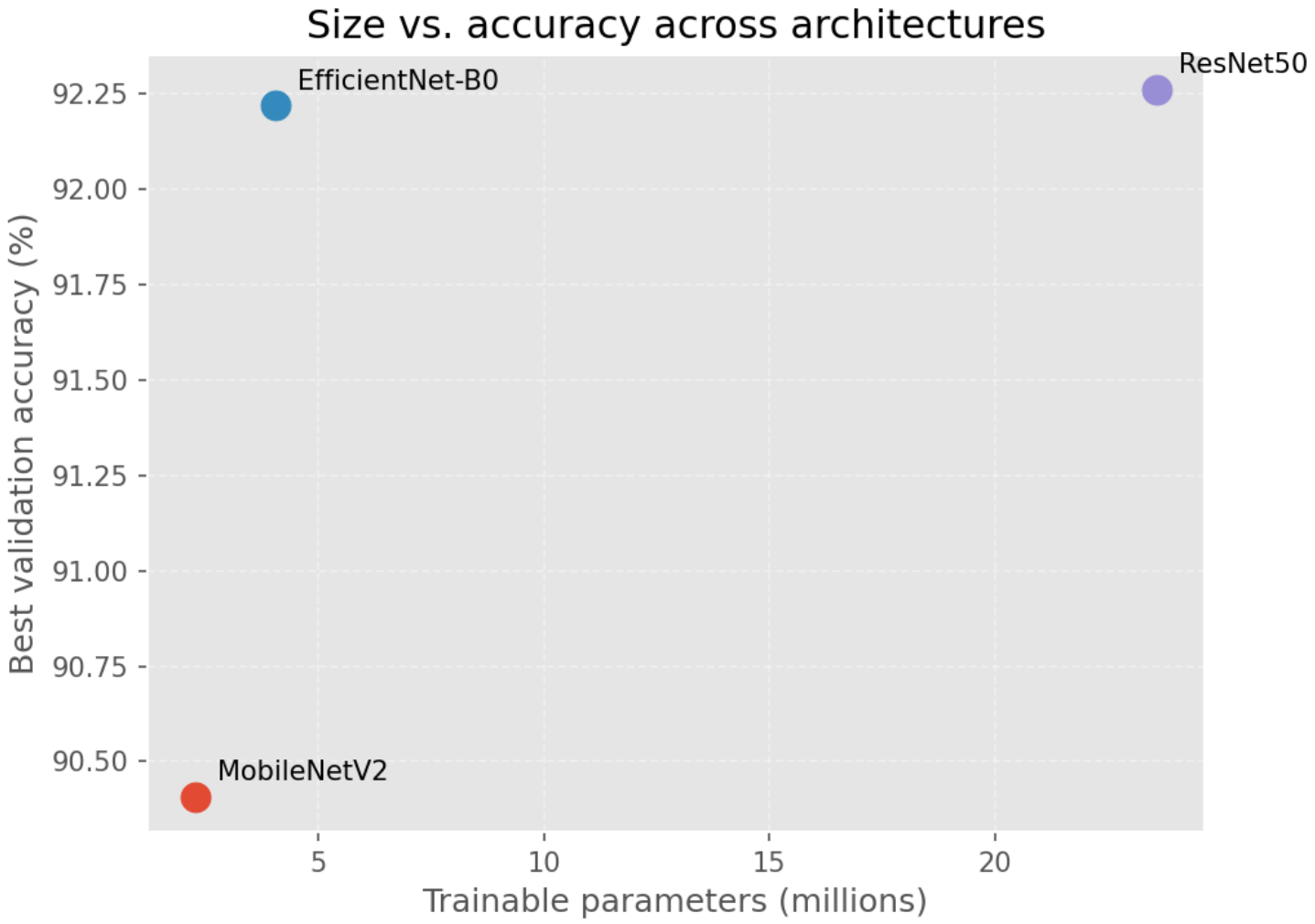


ResNet



Results: Size vs. Accuracy

EfficientNet-B0 matches ResNet50 within 0.5% -- at less than 1/5 the parameters.



Arch	Params	Best val	Test top-1	Test top-5
MobileNet V2	2.3M	90.41%	91.06%	99.25%
EfficientNet -B0	4.1M	92.22%	92.22%	99.46%
ResNet50	23.6M	92.26%	92.72%	99.29%

Reference (torchvision ImageNet, 1000 classes):
MobileNetV2 ~72% EfficientNet-B0 ~77% ResNet50 ~76%. All three reach >90% here -- transfer learning carries the load.

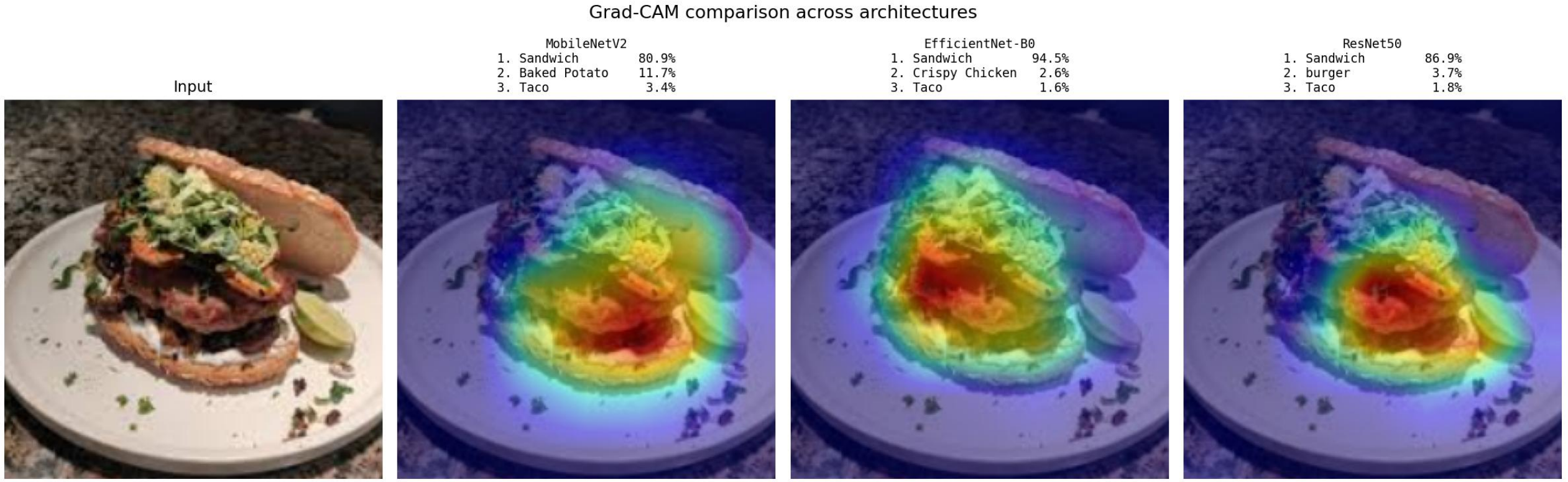
Grad-Cam

- Visual explanation of model decisions
- Highlights important regions in an image
- Shows where the model is “looking”
- Helps interpret and debug model behavior
- Validates that the model focuses on relevant features

- Grad-CAM shows which parts of an image influence the model’s prediction, helping us understand and trust the model’s behavior.

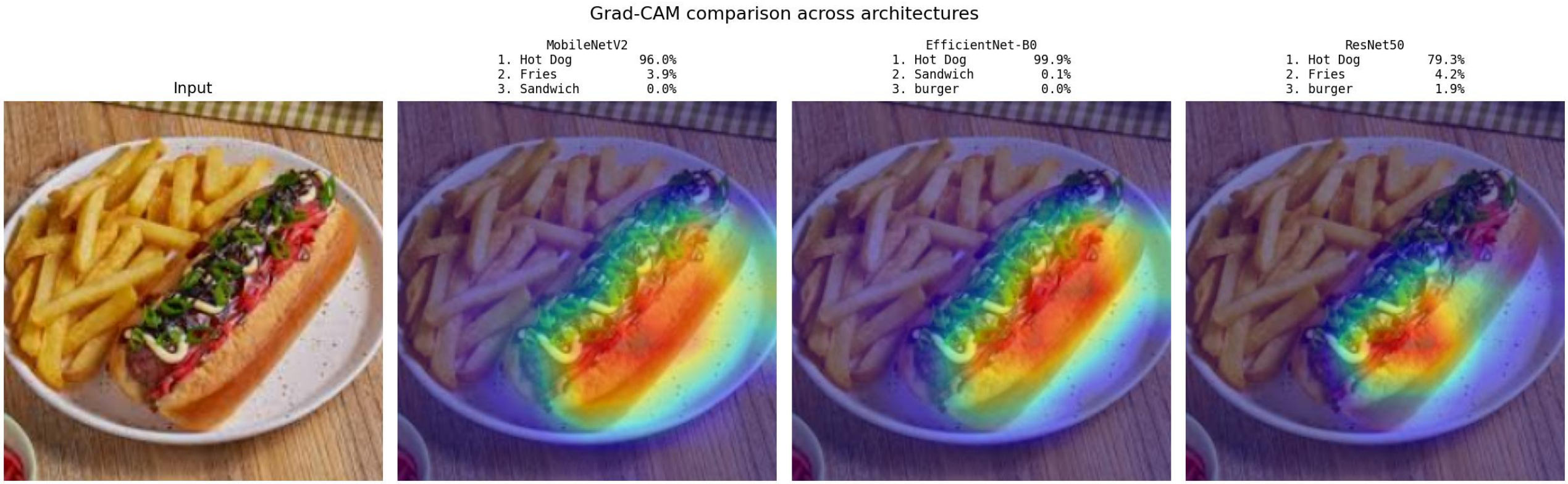
Where the Models Look (Grad-CAM)

- Same image evaluated across all models top-3 predictions per model.
- Models focus on the main food region
- Top-1 prediction shows final output; Top-3 reveals uncertainty
- Confusion occurs between visually similar classes
- Attention patterns vary slightly across architectures
- All models focus correctly, but differ in confidence and attention spread

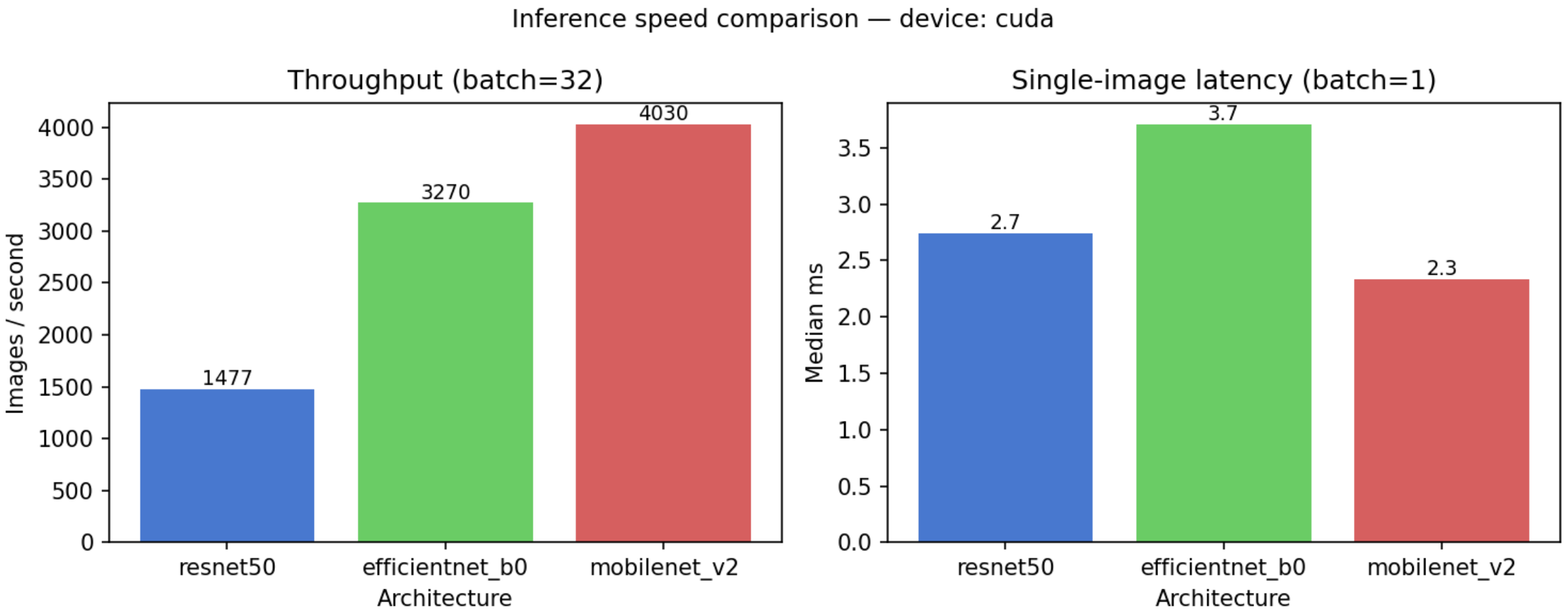


Where the Models Look (Grad-CAM)

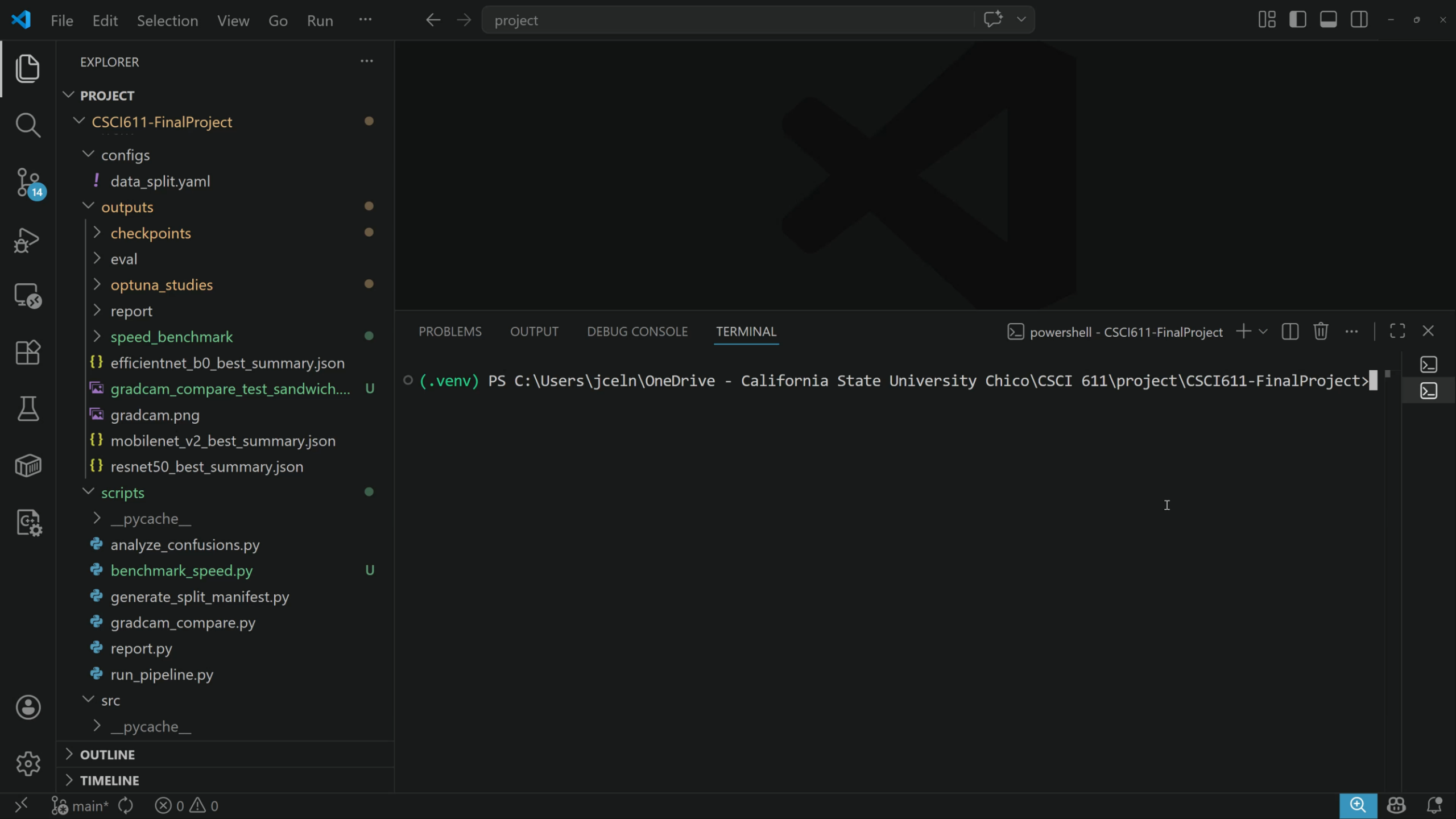
- Image contains multiple food items (hot dog + fries) and All models focus on the hot dog (dominant object)
- MobileNet & ResNet assign small probability to fries (3–4%)
- EfficientNet shows higher confidence in hot dog, minimal fries probability
- Models differ in how they distribute confidence across secondary classes
- Limitation of single-label classification: predicts dominant object only



Speed Comparison



DEMO:
Run grad-CAM on test images from the wild


File Edit Selection View Go Run
project
EXPLORER
PROJECT
CSCI611-FinalProject
configs
data_split.yaml
outputs
checkpoints
eval
optuna_studies
report
speed_benchmark
efficientnet_b0_best_summary.json
gradcam_compare_test_sandwich....
gradcam.png
mobilenet_v2_best_summary.json
resnet50_best_summary.json
scripts
__pycache__
analyze_confusions.py
benchmark_speed.py
generate_split_manifest.py
gradcam_compare.py
report.py
run_pipeline.py
src
__pycache__
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
powershell - CSCI611-FinalProject
(.venv) PS C:\Users\jceln\OneDrive - California State University Chico\CSCI 611\project\CSCI611-FinalProject>
main*
0 0

Questions?

Thank you for watching